

Agentic Synth

High-Level Architecture Document

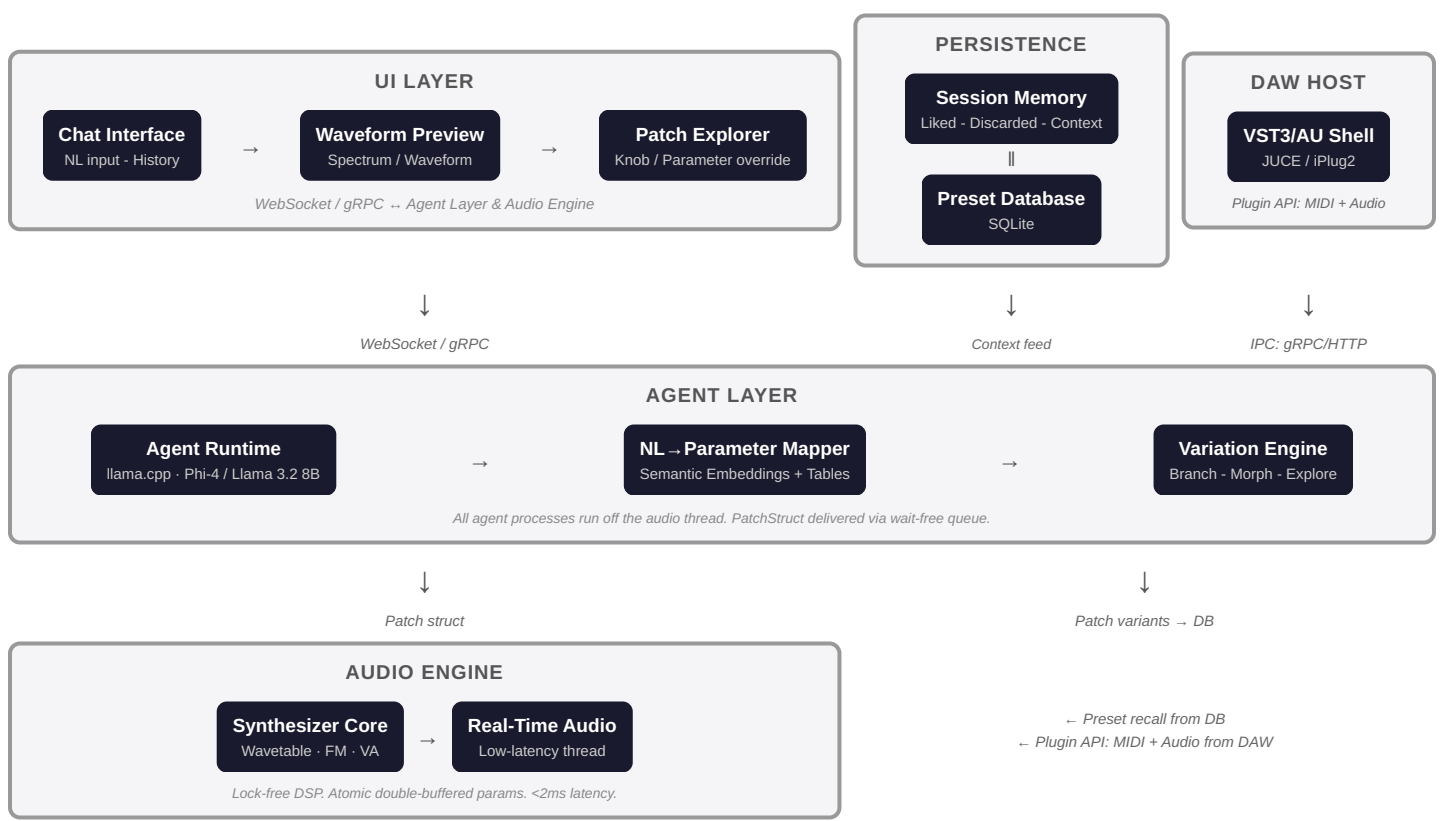
VST3/AU Plugin + Desktop App | Agent-Driven Sound Design

1. Overview

Agentic Synth is a VST3/AU plugin and companion desktop app that lets music producers describe sounds in natural language and receive synthesizer patches in real time. A producer speaks or types a description like "a dark, ominous bass with rhythmic earthquake-like vibrations" and an on-device AI agent translates that into oscillator configs, filter settings, envelopes, and LFO routing — then renders the patch immediately.

Core value: Close the gap between sonic imagination and patch creation. The agent understands musical semantics, remembers liked/discarded sounds, generates variations, and adjusts when you say "make it wider."

2. System Architecture



3. Component Descriptions

VST3/AU Shell

- **Framework:** JUCE 7 (preferred) or IPlug2
- Exposes the plugin to any DAW via VST3 and AU APIs. Owns the audio callback — the only component on the real-time thread.
- Bridges MIDI/automation events into the audio engine. Launches companion app via local gRPC or named pipe.

Synthesizer Engine

- Hybrid wavetable + FM + virtual-analog core in C++. Wavetable: 256-sample, morphable. FM: 4-6 operator. VA: Moog-ladder/SVF filters.
- All DSP lock-free on the audio thread. Parameter changes via atomic double-buffered structs.

Agent Runtime

- **Inference:** llama.cpp with GGUF quantized models, CPU + Metal/CUDA offload.
- **Models:** Phi-4 (14B Q4_K_M ~8GB VRAM) or Llama 3.2 8B (Q4_K_M ~5GB).
- Separate process from audio thread. Exposes gRPC: `GeneratePatch(description)` and `RefinePatch(patch, delta)`.

NL-to-Synth-Parameter Mapper

- The core glue: converts agent output → `PatchStruct`. Agent outputs structured JSON (osc config, filter, ADSR, LFO, effects).
- Semantic tags (`"warm"` , `"dark"`) resolve via embedding lookup → delta vectors in parameter space. Handles context ambiguity ("dark" for bass vs pad).

Audio Rendering Pipeline

- Runs on the real-time thread. Reads `PatchStruct` from wait-free SPSC queue. <2ms parameter-to-audio latency.

UI Layer

- React + TypeScript + Vite + WebSocket (recommended) or native JUCE UI.
- Chat panel with text/speech input (whisper.cpp for offline STT). Patch explorer, waveform/spectrum viz, preset browser.

Session & Preset DB

- SQLite at `~/.agentic-synth/session.db`. Tables: `patches` , `session_events` , `embeddings` .
- Session context feeds agent prompt. Liked patches boost neighborhood; discarded patches suppressed.

Variation Engine

- Generates N variations (default 5) via temperature-varied prompts, param perturbation ($\pm 15\%$), or interpolation between patches.

4. Data Flow

"A dark evolving pad" — end-to-end trace:

```
1. INPUT → User: "a dark evolving pad"
2. STT → whisper.cpp transcribes (if speech)
3. AGENT → llama.cpp produces JSON patch spec:
  { osc1: {type: wavetable, detune: 8},
    osc2: {type: VA, saw, octave: -1},
    filter: {type: SVF_LP, cutoff: 800, resonance: 0.3},
    env: {attack: 1200ms, decay: 400ms, sustain: 0.7, release: 2800ms},
    lfo1: {target: filter_cutoff, rate: 0.25Hz, depth: 0.4},
    lfo2: {target: wavetable_pos, rate: 0.08Hz, depth: 0.6},
    reverb: {size: 0.85, damp: 0.4} }
4. MAPPER → Semantic tags: "dark"→filter -30%, "evolving"→LFO +20%
5. QUEUE → PatchStruct pushed to wait-free queue
6. AUDIO → Synth renders. Waveform preview → UI via WebSocket
7. USER HEARS → ~50-200ms total latency
8. TWEAK → "Make filter more open, add rhythmic gating"
9. REFINE → Agent updates filter.cutoff (+40%), adds trem LFO
10. MEMORY → Session logs patch_id, description, feedback
```

5. Technology Choices

Component	Choice	Rationale
Audio framework	JUCE 7 (C++)	Battle-tested VST3/AU/standalone, large community, commercial license
Agent inference	llama.cpp	CPU + Metal/CUDA, GGUF quantization, proven real-time performance
LLM model	Phi-4 14B Q4 / Llama 3.2 8B Q4	Best instruction-following per size. Runs on 2022+ hardware
Parameter mapping	Semantic embeddings + tables	LLM output too noisy alone; tables provide guardrails
UI	React + TS + Vite + WebSocket	Richer chat UX, faster iteration than native JUCE
IPC	gRPC (local)	Type-safe, low overhead, bidirectional streaming
Persistence	SQLite	Zero-dependency, embedded, sufficient for this scale
Speech input	whisper.cpp (offline)	Privacy-first, no API cost, runs alongside agent

6. Key Risks & Mitigations

Real-Time Audio Safety

Risk: Agent inference blocks audio thread → dropouts.

Mitigation: Agent in separate process. Audio reads from wait-free queue only. Double-buffered params. Agent never on audio call stack.

Harmful Frequency Generation

Risk: Extreme param values (0Hz filter, DC offset).

Mitigation: All values clamped in Mapper before reaching synth. Hard limits in PatchStruct validator. Resonance capped.

NL Inference Latency

Risk: 2-5s lag before hearing a patch.

Mitigation: Persistent KV cache, speculative decoding, streaming partial patches, instant preview mode (<200ms heuristic pre-patch while full inference completes).

Semantic Ambiguity

Risk: "Dark" means different things per context/user.

Mitigation: Domain-grounded system prompt. Session history disambiguates. Clarifying question if low confidence. User-editable semantic dictionary.

7. Implementation Phases

Phase 1 — Standalone App (Foundation)

JUCE standalone + wavetable/VA core. Hardcoded ~50 descriptors → presets. Basic React UI. SQLite storage. No LLM yet.

Deliverable: Type 20-30 descriptors, hear a reasonable patch.

Phase 2 — Local LLM + Chat

Integrate llama.cpp with Phi-4/Llama. Build mapper with embeddings. gRPC bridge. Chat + streaming. Variation engine (5 variants). Session memory.

Deliverable: Full NL patch generation with session awareness and variations.

Phase 3 — VST3/AU Plugin

JUCE wrapper around synth core. Plugin ↔ app IPC. MIDI/automation handling. DAW save/restore. Real-time safety audit.

Deliverable: Runs stably in Ableton Live and Logic Pro.

Phase 4 — Advanced Agentic Features

Style transfer ("make this like [reference]"). Session-aware generation. Smart morphing (MIDI-driven interpolation). Multi-modal input (hum/tap → timbre). Collaborative iteration. Export to Serum/Vital formats.

Deliverable: A creative tool that feels like a sound designer in the session.